

scheduler — fiche d'utilisation

Composant A-Frame qui orchestre dans le temps les events des librairies `effects-model.js` et `effects-vertices.js` (ou tout autre event A-Frame). Il combine deux couches :

- une **timeline** scriptée : des cues à des instants précis, pour un scénario fixe (façon partition).
- une couche **random** : des règles périodiques qui, à intervalle irrégulier, tirent un event au sort dans un pool et le déclenchent.

Une seule entité "chef d'orchestre" porte le composant `scheduler` ; elle n'a pas besoin de porter elle-même un modèle 3D. Chaque cue précise sa propre cible (`target`), ce qui permet d'orchestrer plusieurs objets de la scène depuis un seul scénario.

Mise en place minimale

```
<script type="application/json" id="scenario">
{
  "target": "#chair",
  "timeline": [
    { "time": 0,    "event": "wireframe" },
    { "time": 8000, "event": "vertices" }
  ],
  "random": [
    { "events": ["vertices-glitch"], "interval": 6000, "jitter": 0.5 }
  ]
}
</script>

<a-entity id="director"
  scheduler="config: #scenario; duration: 60000; loop: false; autostart: true">
</a-entity>
```

Attributs du composant

Attribut	Type	Défaut	Rôle
config	string	''	Sélecteur (#id d'un <code><script type="application/json"></code>) ou JSON inline directement dans l'attribut.
duration	number	60000	Durée totale du scénario, en ms. À 0, pas de fin automatique.
loop	boolean	false	Si true, relance un nouveau cycle dès que <code>duration</code> est atteint.
autostart	boolean	true	Démarre automatiquement au chargement de la scène. À false, il faut envoyer l'event <code>start</code> .

Events écoutés (sur l'entité qui porte scheduler)

Event

Effet

`start` (Re)démarre le scénario depuis 0.
`stop` Arrête et remet l'horloge à 0 au prochain `start`.
`pause` Suspend l'avancement de l'horloge (les cues déjà tirées le restent).
`resume` Reprend là où c'était, sans saut de temps.

Event émis

Event

Quand

`scheduler-ended` Émis sur l'entité `scheduler` quand `duration` est atteint et `loop: false`.

Format du JSON de configuration

```
{
  "target": "#chair",
  "timeline": [
    { "time": 0,      "event": "wireframe" },
    { "time": 20000, "target": "#chair2", "event": "vertices" }
  ],
  "random": [
    { "target": "#chair", "events": ["vertices-glitch", "vertices-corrupt"],
      "interval": 6000, "jitter": 0.3 }
  ]
}
```

- **target** (racine, optionnel) : sélecteur CSS par défaut (`#id` ou `.classe` — une classe envoie l'event à tous les éléments trouvés). Sert quand une cue ou une règle `random` ne précise pas la sienne.
 - **timeline** (tableau de *cues*) : chaque cue = { `time`, `target?`, `event` }.
 - `time` : instant de déclenchement en ms depuis le `start`.
 - `target` : cible spécifique à cette cue (sinon celle par défaut).
 - `event` : nom de l'event A-Frame à émettre (ex. `wireframe`, `vertices-erosion`...).
 - Une cue se déclenche **une seule fois** par cycle, à sens unique.
 - **random** (tableau de règles) : chaque règle = { `target?`, `events`, `interval`, `jitter` }.
 - `events` : liste d'events possibles ; un seul est tiré au sort à chaque déclenchement. Avec un seul event dans la liste, ça revient à un simple déclenchement périodique irrégulier.
 - `interval` : durée moyenne entre deux déclenchements, en ms.
 - `jitter` : irrégularité, en proportion de `interval` (0 = métronomique, 0.5 = ±50 %).
-

Qu'est-ce qu'une "cue" ?

Terme emprunté au théâtre : le signal qui indique *quand* et *quoi* déclencher. Ici, une cue de la `timeline` est un triplet **instant / cible / event**.

⚠ Point d'attention : les toggles

La plupart des events des 2 librairies sont des **toggles** — un 2e envoi annule le 1er (`wireframe`, `transparent`, `desappear`, `bluescreen`, `reboot`, `delete`, `vertices`, `vertices-desappear`, `vertices-falling`, `vertices-binary-rain`...). Le `scheduler` se contente d'émettre l'event demandé, sans connaître son état côté effet. Si une cue rappelle un event déjà déclenché plus tôt dans le même cycle, elle l'annule au lieu de le redéclencher. À anticiper en écrivant le scénario (et à surveiller particulièrement avec `loop: true`, où un cycle qui se termine dans un état "activé" repart désactivé au cycle suivant, ou inversement).

Quelques events ne sont **pas** des toggles (aller simple) : `vertices-glitch`, `vertices-corrupt`, `vertices-scatter`, `vertices-erosion` — ceux-là peuvent être redéclenchés sans risque de s'annuler.

Exemple multi-objets

```
<script type="application/json" id="scenario">
{
  "timeline": [
    { "time": 0,      "target": "#chair1", "event": "wireframe" },
    { "time": 5000,   "target": "#chair2", "event": "vertices" },
    { "time": 20000,  "target": "#chair2", "event": "vertices-erosion" },
    { "time": 35000,  "target": "#chair1", "event": "desappear" }
  ],
  "random": [
    { "target": "#chair1", "events": ["wireframe"],      "interval": 10000, "jitter": 0.4 },
    { "target": "#chair2", "events": ["vertices-glitch"], "interval": 6000,  "jitter": 0.5 }
  ]
}
</script>
```

Pas de `target` par défaut ici : chaque cue et chaque règle précise la sienne. Ajouter une `#chair3` ne demande aucune modification du composant, juste de nouvelles cues dans le JSON.

Limites connues

- Une cue non trouvée (`target` introuvable au moment du déclenchement) produit un `console.warn`, pas d'erreur bloquante.
- Le JSON est relu une seule fois à l'`init` du composant ; changer l'attribut `config` à chaud ne recharge pas le scénario (il faudrait détruire/recréer l'entité, ou étendre le composant avec un `update()`).

